MongoDB Document Store

Chapter 06 - Sub-Chapter 05

NoSQL Databases Series

What is MongoDB?	World's most popular document database
CRUD Operations	insertOne, find, update, delete with Java
Spring Data MongoDB	Repositories, @Document, MongoTemplate
Aggregation Pipeline	$match, $group, $sort, $lookup
Indexing & Performance	Indexes, explain(), TTL, text search
When to Use MongoDB	Flexible schemas, nested data, content

1 What is MongoDB?

The world's most popular document database

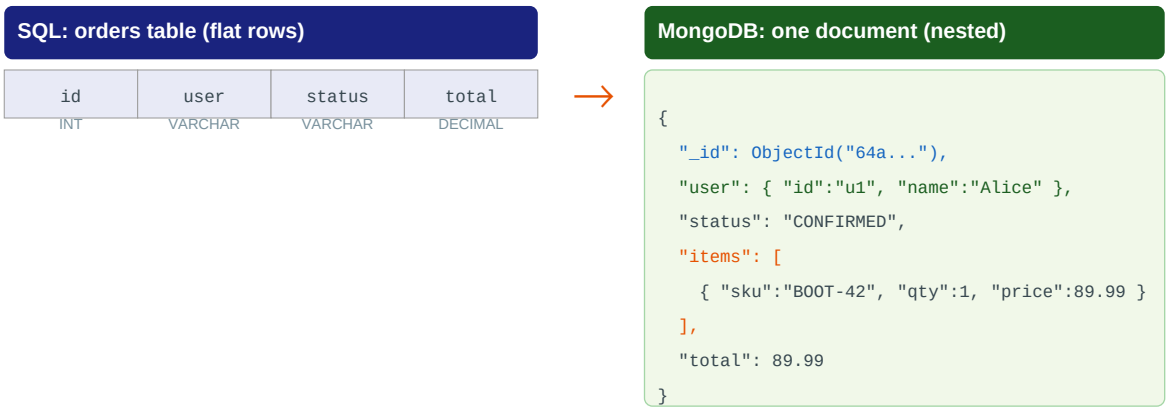
Relational databases store data in rows and columns — a rigid grid. But the real world does not fit neatly into grids. An order has a customer, line items, a shipping address, payment info — data that is naturally nested, not flat. MongoDB embraces this natural shape.

MongoDB stores data as **BSON documents** (Binary JSON) — flexible, schema-free records that can contain nested objects and arrays. Launched in 2009, it is now the most widely used NoSQL database, deployed by LinkedIn, Forbes, Expedia, and hundreds of thousands of applications.

Why developers love MongoDB:

- **Natural data model:** Your Java objects map directly to documents — no ORM impedance mismatch.
- **Schema flexibility:** Each document can have different fields. Add a new attribute without ALTER TABLE.
- **Rich query language:** Filter, project, sort, aggregate, do geospatial queries — all built-in.
- **Horizontal scaling:** Sharding distributes data across multiple servers automatically.
- **Full ACID transactions:** Multi-document, multi-collection transactions supported since MongoDB 4.0.
- **Atlas cloud:** MongoDB Atlas is a fully managed cloud service with free tier — zero-friction start.

MongoDB Document — Nested, Flexible, Self-Describing



2 CRUD Operations

insertOne, find, updateOne, deleteOne — with Java

Let us build a simple order management system. We will use the MongoDB Java Driver directly first, then show Spring Data MongoDB on top.

Setup and connection:

```
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import org.bson.Document;

// Connect to MongoDB (local or Atlas)
MongoClient client = MongoClients.create(
    "mongodb+srv://user:pass@cluster0.mongodb.net/?retryWrites=true"
);
MongoDatabase db   = client.getDatabase("ecommerce");
MongoCollection<Document> orders = db.getCollection("orders");
```

Insert a document:

```
import org.bson.Document;
import java.util.Arrays;
import java.util.Date;

Document order = new Document("userId", "alice")
    .append("status", "CONFIRMED")
    .append("total", 120.50)
    .append("createdAt", new Date())
    .append("items", Arrays.asList(
        new Document("sku", "BOOT-42").append("qty", 1).append("price", 89.99),
        new Document("sku", "SOCK-L").append("qty", 2).append("price", 15.25)
    ))
    .append("shippingAddress", new Document("street", "123 Main St")
        .append("city", "New York").append("zip", "10001"));

orders.insertOne(order);
// MongoDB auto-generates _id: ObjectId("64a7f2b3c1234...")
System.out.println("Inserted: " + order.getObjectId("_id"));
```

Find documents — basic queries:

```
import com.mongodb.client.FindIterable;
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Sorts;
import com.mongodb.client.model.Projections;

// Find all orders for alice
FindIterable<Document> aliceOrders = orders.find(
    Filters.eq("userId", "alice")
).sort(Sorts.descending("createdAt")).limit(10);

// Find confirmed orders over $100
orders.find(
    Filters.and(
        Filters.eq("status", "CONFIRMED"),
        Filters.gt("total", 100.0)
```

```
    )
).forEach(doc -> System.out.println(doc.toJson()));

// Find with projection (only return specific fields)
orders.find(Filters.eq("userId", "alice"))
    .projection(Projections.include("status", "total", "createdAt"))
    .forEach(doc -> System.out.println(doc.toJson()));
```

Update and Delete:

```
import com.mongodb.client.model.Updates;
import com.mongodb.client.result.UpdateResult;
import com.mongodb.client.result.DeleteResult;
import org.bson.types.ObjectId;

// Update status of one order (by _id)
UpdateResult result = orders.updateOne(
    Filters.eq("_id", new ObjectId("64a7f2b3c1234...")),
    Updates.combine(
        Updates.set("status", "SHIPPED"),
        Updates.currentDate("updatedAt"),
        Updates.push("trackingEvents", new Document("event", "DISPATCHED").append("ts", new Date()
))
    )
);
System.out.println("Modified: " + result.getModifiedCount());

// Delete orders older than 1 year
DeleteResult del = orders.deleteMany(
    Filters.lt("createdAt", new Date(System.currentTimeMillis() - 365L*24*3600*1000))
);
System.out.println("Deleted: " + del.getDeletedCount());
```

3 Spring Data MongoDB
Repositories and MongoTemplate

Spring Data MongoDB wraps the driver and gives you the same @Repository and @Document experience you expect from the Spring ecosystem. Zero boilerplate for common operations.

Maven dependency:

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

application.yml:

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://user:pass@cluster0.mongodb.net/ecommerce
      # OR for local:
      host: localhost
      port: 27017
      database: ecommerce
```

Document entity class:

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;

@Document(collection = "orders")    // maps to 'orders' collection
public class Order {

    @Id
    private String id;              // maps to MongoDB _id (ObjectId as String)

    @Indexed                        // creates a MongoDB index
    @Field("userId")
    private String userId;

    private String status;
    private BigDecimal total;
    private Instant createdAt;
    private List<OrderItem> items;  // embedded sub-documents (no join needed!)
    private Address shippingAddress;

    // inner classes
    public static class OrderItem {
        private String sku;
        private int qty;
        private BigDecimal price;
        // getters/setters
    }
    public static class Address {
```

```
        private String street, city, zip;
        // getters/setters
    }
    // getters/setters/constructors
}
```

Repository interface:

```
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.data.mongodb.repository.Query;
import org.springframework.stereotype.Repository;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;

@Repository
public interface OrderRepository extends MongoRepository<Order, String> {

    // Spring Data derives the query from the method name
    List<Order> findByUserId(String userId);

    List<Order> findByUserIdAndStatus(String userId, String status);

    List<Order> findByTotalGreaterThan(BigDecimal minTotal);

    // Custom MongoDB query with @Query
    @Query("{ 'userId': ?0, 'createdAt': { $gte: ?1 } }")
    List<Order> findRecentOrders(String userId, Instant since);

    long countByStatus(String status);
}
```

Service using the repository:

```
import org.springframework.stereotype.Service;
import java.math.BigDecimal;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.List;

@Service
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public Order createOrder(String userId, List<Order.OrderItem> items) {
        Order order = new Order();
        order.setUserId(userId);
        order.setItems(items);
        order.setStatus("PENDING");
        order.setTotal(items.stream()
            .map(i -> i.getPrice().multiply(BigDecimal.valueOf(i.getQty())))
            .reduce(BigDecimal.ZERO, BigDecimal::add));
        order.setCreatedAt(Instant.now());
        return orderRepository.save(order);    // inserts into MongoDB
    }
```

```
    public List<Order> getLastMonthOrders(String userId) {
        Instant oneMonthAgo = Instant.now().minus(30, ChronoUnit.DAYS);
        return orderRepository.findRecentOrders(userId, oneMonthAgo);
    }
}
```

4 Aggregation Pipeline

The most powerful feature of MongoDB

The aggregation pipeline is MongoDB's version of SQL GROUP BY, JOIN, and HAVING — but far more expressive. Documents flow through a sequence of stages, each transforming the data. Think of it as an assembly line for data.

Aggregation Pipeline — Documents Flow Through Stages

Scenario: Sales dashboard — revenue by category, top customers

```
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.aggregation.*;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class SalesAnalyticsService {

    private final MongoTemplate mongoTemplate;

    public SalesAnalyticsService(MongoTemplate mongoTemplate) {
        this.mongoTemplate = mongoTemplate;
    }

    // Total revenue per status, only CONFIRMED orders this month
    public List<StatusRevenue> revenueByStatus() {
        MatchOperation match = Aggregation.match(
            Criteria.where("status").is("CONFIRMED")
                    .and("createdAt").gte(Instant.now().minus(30, ChronoUnit.DAYS))
        );
        GroupOperation group = Aggregation.group("status")
            .count().as("orderCount")
            .sum("total").as("totalRevenue");
        SortOperation sort = Aggregation.sort(
            Sort.by(Sort.Direction.DESC, "totalRevenue")
        );
        Aggregation pipeline = Aggregation.newAggregation(match, group, sort);

        return mongoTemplate
            .aggregate(pipeline, "orders", StatusRevenue.class)
            .getMappedResults();
    }
}

// Result DTO
public class StatusRevenue {
    private String id;         // the group-by field (status)
    private long orderCount;
```

```
    private double totalRevenue;
    // getters/setters
}
```

$lookup — joining collections (like SQL JOIN):

```
// Join orders with products to get product details
// MongoDB: use $lookup (use sparingly — embedding is usually better)

// Via MongoTemplate
LookupOperation lookup = Aggregation.lookup(
    "products",         // from collection
    "items.sku",        // localField
    "sku",              // foreignField
    "productDetails"    // as
);

Aggregation pipeline = Aggregation.newAggregation(
    Aggregation.match(Criteria.where("userId").is("alice")),
    lookup,
    Aggregation.project("status", "total", "productDetails")
);

// In raw MQL (for reference):
// db.orders.aggregate([
//   { $match: { userId: "alice" } },
//   { $lookup: { from:"products", localField:"items.sku",
//                foreignField:"sku", as:"productDetails" } }
// ])
```

5 Indexing and Performance

Query plans, explain, compound indexes

Without indexes, MongoDB performs a collection scan — it reads every document to find matches. On a collection with millions of documents, this is catastrophically slow. Indexes are your most important performance tool.

Creating indexes (Java driver):

```
import com.mongodb.client.model.IndexOptions;
import com.mongodb.client.model.Indexes;

// Single field index -- fast lookup by userId
orders.createIndex(Indexes.ascending("userId"));

// Compound index -- supports queries filtering on both fields
orders.createIndex(Indexes.compoundIndex(
    Indexes.ascending("userId"),
    Indexes.descending("createdAt")
));

// Unique index -- enforce uniqueness (like SQL UNIQUE constraint)
orders.createIndex(Indexes.ascending("orderRef"),
    new IndexOptions().unique(true));

// TTL index -- auto-delete documents after X seconds
orders.createIndex(Indexes.ascending("createdAt"),
    new IndexOptions().expireAfter(90L, TimeUnit.DAYS));  // auto-delete after 90 days

// Text index -- full-text search
orders.createIndex(Indexes.text("description"));
orders.find(Filters.text("running shoes")).forEach(d -> System.out.println(d.toJson()));
```

Explain a query — check if index is used:

```
// In MongoDB shell or Compass -- always check this before going to production!
db.orders.find({ userId: "alice", status: "CONFIRMED" }).explain("executionStats")

// Look for:
//   "stage": "IXSCAN"   <-- good, using index
//   "stage": "COLLSCAN" <-- bad, full table scan -- add an index!
//   "nReturned": 5, "totalKeysExamined": 5  <-- perfect, no wasted work
//   "nReturned": 5, "totalDocsExamined": 50000  <-- bad, examine 50k to find 5

// Rule of thumb: totalDocsExamined should be close to nReturned
```

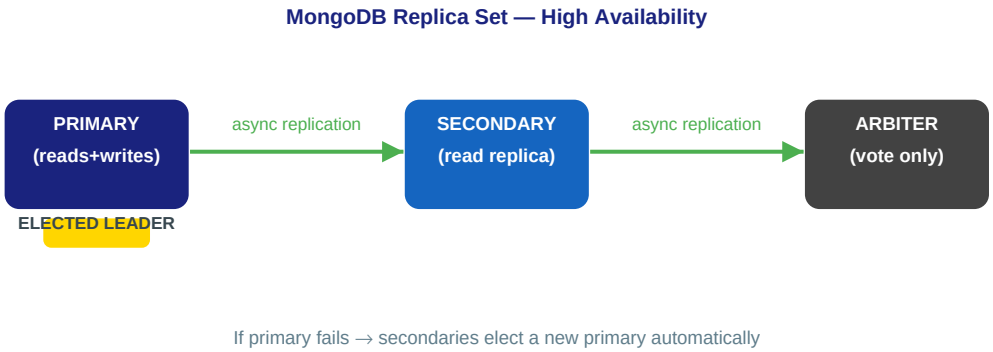
Spring Data MongoDB index annotations:

```
import org.springframework.data.mongodb.core.index.CompoundIndex;
import org.springframework.data.mongodb.core.index.CompoundIndexes;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "orders")
@CompoundIndexes({
    @CompoundIndex(name = "user_date_idx",
                   def = "{'userId': 1, 'createdAt': -1}")  // supports sorting
})
public class Order {
```

```
    @Indexed(unique = true)
    private String orderRef;           // unique index

    @Indexed(expireAfterSeconds = 7776000)  // TTL: 90 days
    private Instant tempToken;
    // ...
}
```

6 When to Use MongoDB

The strengths, the limits, the decision



MongoDB excels at:

✓ **Flexible, evolving schemas:** Startup moving fast, product catalog with varied attributes, user-generated content.

✓ **Hierarchical / nested data:** Orders with line items, blog posts with comments, user profiles with addresses.

✓ **Content management:** Articles, product pages, configuration data — each with different shapes.

✓ **Real-time analytics on operational data:** Aggregation pipeline runs close to the data, no ETL needed.

✓ **Full-text search:** Built-in text indexes. Atlas Search adds Lucene-based search on top.

MongoDB is a poor fit for:

✗ **Highly relational data with many JOINs:** If your data is naturally tabular with lots of foreign keys, use PostgreSQL.

✗ **Pure time-series workloads:** Cassandra or a dedicated TSDB (InfluxDB) handle billions of time-stamped records more efficiently.

✗ **Strong consistency across many entities by default:** Transactions are supported but add overhead — design to minimize them.

Company	What they use MongoDB for	Why not SQL?
LinkedIn	User profiles, connections metadata	Schema evolves constantly
Forbes	Article content, author profiles	Variable article structure
Expedia	Hotel/flight inventory catalog	Millions of attribute variants
Bosch	IoT sensor configuration, device logs	Nested, varied device schemas
eBay	Product catalog, search indexes	50M+ product types, each unique

Key Rules

- Embed documents when data is read together — avoid $lookup like a SQL JOIN (it is slower).
- Use references (ObjectIds) only when the related data is large or updated independently.

- Index every field you query on — run explain() to verify IXSCAN not COLLSCAN.
- Design for your most frequent query first — MongoDB is query-pattern driven, not entity-driven.
- Use Atlas Search for full-text search rather than building your own Elasticsearch cluster.